

Introduzione al Linguaggio Python

Python è un linguaggio di programmazione **interpretabile, dinamico e ad alto livello**, noto per la sua **semplicità e leggibilità**. Grazie alla sua sintassi chiara e intuitiva, è uno dei linguaggi più utilizzati sia dai principianti che dagli esperti per una vasta gamma di applicazioni, tra cui sviluppo web, automazione, intelligenza artificiale, analisi dati e molto altro.

Concetti Fondamentali in Python

1. Linguaggi Compilati ed Interpretati in Python

- **Python è un linguaggio interpretato**, il che significa che il codice viene eseguito riga per riga da un interprete, senza essere tradotto in codice macchina prima dell'esecuzione.
- **Differenza con i linguaggi compilati:** Nei linguaggi compilati (come C o C++), il codice sorgente viene tradotto in un file eseguibile prima di essere eseguito. Python, invece, viene interpretato direttamente, il che lo rende più portabile ma generalmente più lento.
- Python utilizza una compilazione intermedia: il codice sorgente (.py) viene convertito in bytecode (.pyc), che poi viene eseguito dall'interprete Python.

2. Linguaggio Gestito, Semi-Gestito e Non Gestito in Python

- **Python è un linguaggio gestito**, il che significa che la gestione della memoria è automatizzata dal runtime del linguaggio.
- La gestione della memoria include:
 - **Allocazione dinamica** degli oggetti.
 - **Garbage Collector** che libera automaticamente la memoria occupata da oggetti non più utilizzati.
- **Differenza con linguaggi semi-gestiti e non gestiti:**
 - **Linguaggi semi-gestiti** (es. C++ con smart pointers) offrono sia gestione manuale che automatica della memoria.
 - **Linguaggi non gestiti** (es. C) richiedono la gestione manuale della memoria con malloc e free.

3. Linguaggi Non Tipizzati, Debolmente Tipizzati e Fortemente Tipizzati in Python

- **Python è fortemente tipizzato** perché non permette conversioni implicite tra tipi incompatibili.
`x = "5" + 2` # Errore: TypeError
- **Debolmente tipizzato** sarebbe un linguaggio come JavaScript, che converte implicitamente i tipi:
`let x = "5" + 2; // Risultato: "52"`
- **Linguaggi non tipizzati** (come Assembly) non hanno una distinzione chiara tra i tipi di dati.

4. Linguaggi Procedurali, ad Oggetti e Funzionali in Python

- **Python supporta tutti questi paradigmi:**
 - **Procedurale:** Si basa su funzioni e istruzioni sequenziali.
 - **Orientato agli Oggetti (OOP):** Supporta classi e oggetti.
 - **Funzionale:** Supporta funzioni pure e strumenti come `map`, `filter` e `lambda`.

Esempio di programmazione **funzionale**:

```
numeri = [1, 2, 3, 4]
```

```
quadrati = list(map(lambda x: x**2, numeri))
```

```
print(quadrati) # Output: [1, 4, 9, 16]
```

5. Linguaggi Single-Platform e Multi-Platform in Python

- **Python è multi-platform**, poiché può essere eseguito su Windows, macOS e Linux senza modifiche al codice.
- A differenza di linguaggi come Swift (che è ottimizzato per iOS) o C# (fortemente integrato con Windows), Python è altamente portabile.

6. Libreria Standard di Python

- Python ha una **libreria standard** molto ampia che offre strumenti per:
 - Manipolazione file (`os`, `shutil`).
 - Operazioni matematiche (`math`, `random`).
 - Networking (`socket`, `http.client`).
 - Elaborazione dati (`json`, `csv`).

Esempio con il modulo `math`:

```
import math

print(math.sqrt(16))  # Output: 4.0
```

7. Gestione della Memoria e Python Garbage Collector

- Python utilizza un **Garbage Collector (GC)** per liberare la memoria occupata da oggetti non più referenziati.

Il modulo `gc` permette di interagire con il GC:

```
import gc

gc.collect()  # Forza il rilascio della memoria
```

- **Differenza con linguaggi come C:** In C, la memoria deve essere gestita manualmente con `malloc` e `free`.

8. Interprete Python e CPython

- **L'interprete Python** è il programma che esegue il codice Python.
- **CPython** è l'implementazione principale di Python, scritta in C.
- Altre implementazioni di Python:
 - **PyPy** (più veloce grazie alla compilazione Just-In-Time).
 - **Jython** (integra Python con la JVM).
 - **IronPython** (Python su piattaforma .NET).

9. Python Script

- Un **Python script** è un file `.py` che può essere eseguito direttamente.

Esempio (`script.py`):

```
print("Esecuzione di uno script Python")
```

Per eseguirlo da terminale:

```
python script.py
```

10. Struttura di uno Script Python (`main.py`)

- Il file `main.py` è spesso il punto di ingresso di un programma Python.
- Il costrutto `if __name__ == "__main__"` assicura che il codice venga eseguito solo quando il file è eseguito direttamente, e non quando è importato.

Esempio:

```
def saluta():
```

```
    print("Ciao, Python!")
```

```
if __name__ == "__main__":
```

```
    saluta()
```

- Se esegui `python main.py`, verrà stampato "Ciao, Python!".
- Se `main.py` viene importato in un altro script, il codice dentro `if __name__ == "__main__"` **non verrà eseguito automaticamente**.

Caratteristiche Principali di Python

Semplice e leggibile

Python ha una sintassi pulita e minimale, che lo rende facile da leggere e scrivere rispetto ad altri linguaggi come C o Java.

Esempio:

```
print("Ciao, Mondo!") # Stampa a schermo "Ciao, Mondo!"
```

Tipizzazione dinamica e gestione automatica della memoria

Non è necessario dichiarare il tipo delle variabili: Python lo determina automaticamente.

```
x = 10          # Intero
y = 3.14        # Float
nome = "Alice"  # Stringa
```

Multiplatforma

Python è compatibile con **Windows, macOS e Linux**, rendendolo molto versatile.

Librerie e framework potenti

Python ha un vasto ecosistema di **librerie** e **framework** per ogni esigenza:

- **Analisi dati e AI:** NumPy, Pandas, TensorFlow
- **Sviluppo web:** Django, Flask
- **Automazione e scripting:** Selenium, BeautifulSoup

Multiparadigma

Supporta programmazione:

- **Procedurale**
- **Orientata agli oggetti (OOP)**
- **Funzionale**

Primi passi con Python

Variabili e tipi di dati

```
numero = 5          # Intero
pi = 3.14            # Float
testo = "Python"     # Stringa
valore = True        # Booleano
```

Strutture di controllo

➤ Condizioni (if - elif - else)

```
x = 10
if x > 0:
    print("Positivo")
elif x == 0:
    print("Zero")
```

```
else:
    print("Negativo")
```

➤ Cicli (for e while)

```
for i in range(5):
    print(i)  # Stampa numeri da 0 a 4
```

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Funzioni

Le funzioni permettono di riutilizzare codice in modo modulare.

```
def saluta(nome):
    return f"Ciao, {nome}!"
```

```
print(saluta("Luca"))  # Output: Ciao, Luca!
```

Programmazione a Oggetti (OOP)

Python supporta la **Programmazione Orientata agli Oggetti** (OOP), che aiuta a organizzare il codice in classi e oggetti.

```
class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def saluta(self):
        return f"Ciao, mi chiamo {self.nome} e ho {self.eta} anni."
```

```
p = Persona("Luca", 25)
print(p.saluta())  # Output: Ciao, mi chiamo Luca e ho 25 anni.
```

Gestione delle Eccezioni

Python permette di gestire errori ed eccezioni con try-except.

```
try:
    risultato = 10 / 0
except ZeroDivisionError:
    print("Errore: divisione per zero!")
```

Importare Moduli

Python permette di importare e riutilizzare moduli per estendere le funzionalità.

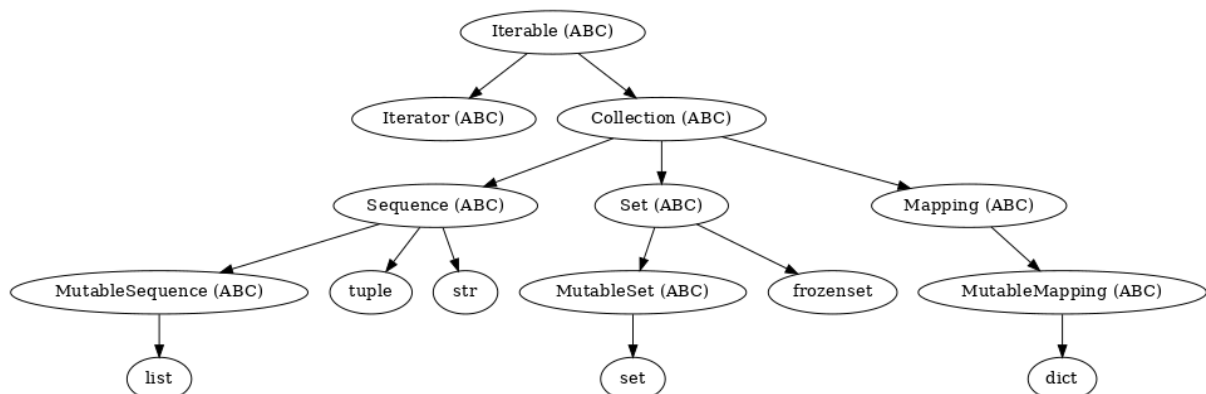
```
import math as mt
print(mt.sqrt(25))  # Output: 5.0
```

Collezioni in Python

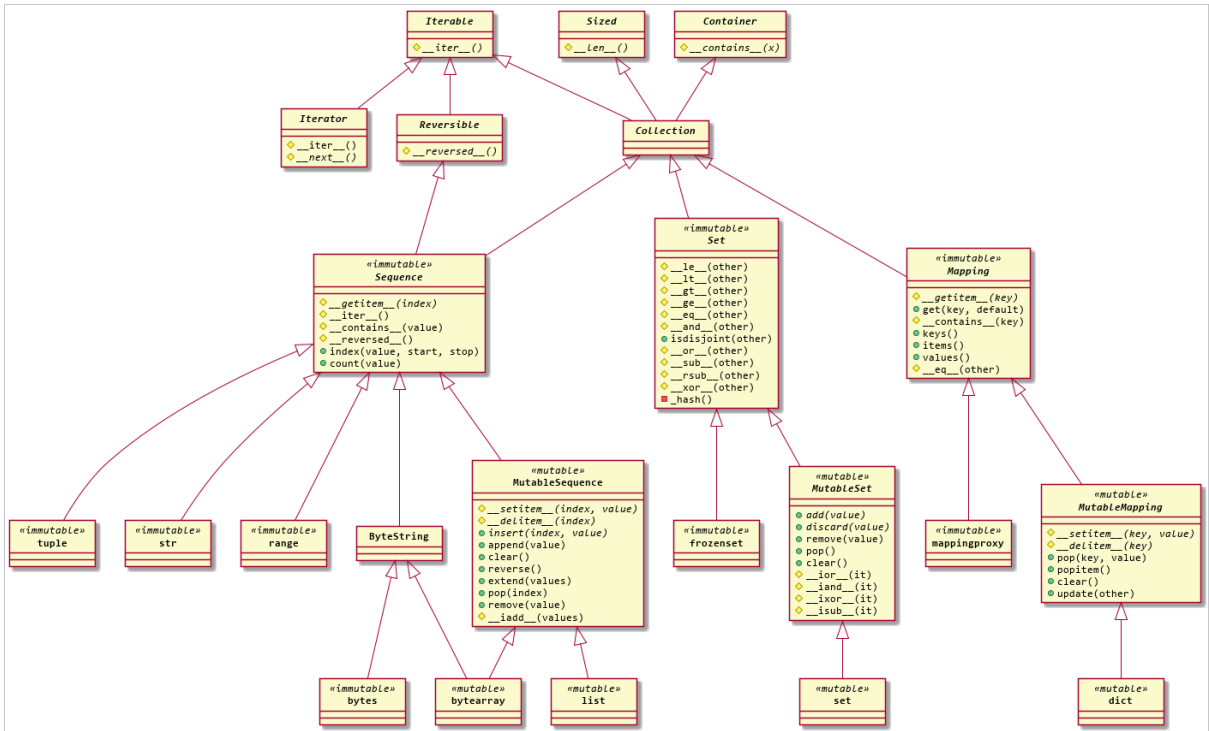
In Python, una collezione è un tipo di dato che permette di raccogliere e gestire più valori all'interno di una singola variabile. Python organizza le collezioni secondo una **gerarchia di classi astratte** definite nel modulo **collections.abc**. Ciascuna classe implementa dei metodi specifici, che servono ad utilizzare e manipolare l'oggetto istanza alla classe. Questa gerarchia aiuta a capire **come le strutture dati sono correlate** e quali **metodi devono implementare** per conformarsi a un'interfaccia comune.

Struttura Gerarchica delle Collezioni in Python

Ecco un diagramma che rappresenta la gerarchia delle collezioni in Python:



Questo schema invece, specializza il diagramma precedente aggiungendo per ciascuna classe concreta o astratta (ABC), i metodi implementati in quella classe:



La gerarchia è composta da **classi astratte (ABC)** che definiscono **metodi minimi** per ogni categoria di collezioni.

Classe	Descrizione	Metodi Chiave
Iterable	Classe base di ogni collezione iterabile in Python.	<code>__iter__()</code>
Collection	Estende Iterable, introduce supporto per <code>len()</code> .	<code>len()</code> , <code>__contains__()</code>
Sequence	Collezioni ordinate e indicizzabili.	<code>__getitem__()</code> , <code>index()</code>
Mapping	Dizionari e strutture chiave-valore.	<code>keys()</code> , <code>values()</code> , <code>items()</code>
MutableMapping	Versione mutabile di Mapping.	<code>__setitem__()</code> , <code>pop()</code>

Vediamo ora le principali collezioni in Python e dove si collocano nella gerarchia.

Iterable (`collections.abc.Iterable`)

Base di tutte le collezioni iterabili:

- Ogni oggetto **iterabile** implementa `__iter__()`, che restituisce un iteratore.
- **Esempi:** `list`, `tuple`, `dict`, `set`, `str`, `generatori`.

```
from collections.abc import Iterable
print(isinstance([1, 2, 3], Iterable))  # True
```

Metodi Disponibili

Metodo	Parametri	Ritorno	Descrizione
<code>__iter__()</code>	<code>self</code>	Iterator	Restituisce un iteratore per l'oggetto.

Esempio

```
from collections.abc import Iterable

class MyIterable:
    def __init__(self, data):
        self.data = data

    def __iter__(self):
        return iter(self.data)  # Restituisce un iteratore sulla
                                # lista interna

my_iterable = MyIterable([1, 2, 3])
print(isinstance(my_iterable, Iterable))  # True

for item in my_iterable:
    print(item)
```

Iterator (`collections.abc.Iterator`)

Oggetto iterabile che implementa anche `__next__()`, permettendo la lettura sequenziale dei dati.

Metodi Disponibili

Metodo	Parametri	Ritorno	Descrizione
<code>__iter__()</code>	<code>self</code>	<code>self</code>	Restituisce l'iteratore stesso.
<code>__next__()</code>	<code>self</code>	Elemento della sequenza	Restituisce il prossimo elemento dell'iterazione, genera <code>StopIteration</code> alla fine.

Esempio

```
from collections.abc import Iterator

class MyIterator:
    def __init__(self, data):
        self.data = data
        self.index = 0

    def __iter__(self):
        return self # Un iteratore è anche Iterable

    def __next__(self):
        if self.index >= len(self.data):
            raise StopIteration # Fine della sequenza
        value = self.data[self.index]
        self.index += 1
        return value

my_iterator = MyIterator([10, 20, 30])
print(isinstance(my_iterator, Iterator)) # True

print(next(my_iterator)) # 10
print(next(my_iterator)) # 20
print(next(my_iterator)) # 30
# print(next(my_iterator)) # StopIteration
```

Collection (`collections.abc.Collection`)

Estende `Iterable`, aggiunge `len()`, in:

- Supporta `__len__()`, `__contains__()`, `__iter__()`.
- **Sottoclassi:** `Sequence`, `Set`, `Mapping`.

```
from collections.abc import Collection
print(isinstance({1, 2, 3}, Collection)) # True
```

Metodi Disponibili

Metodo	Parametri	Ritorno	Descrizione
<code>__iter__()</code>	<code>self</code>	Iteratore	Restituisce un iteratore per l'oggetto.
<code>__len__()</code>	<code>self</code>	<code>int</code>	Restituisce la lunghezza della collezione.
<code>__contains__()</code>	<code>self</code> , <code>item</code>	<code>bool</code>	Controlla se un elemento è nella collezione.

Esempio

```
from collections.abc import Collection

class MyCollection:
    def __init__(self, data):
        self.data = data

    def __iter__(self):
        return iter(self.data)

    def __len__(self):
        return len(self.data)

    def __contains__(self, item):
        return item in self.data

my_collection = MyCollection(["a", "b", "c"])
print(isinstance(my_collection, Collection)) # True
```

```
print(len(my_collection)) # 3
print("b" in my_collection) # True
```

Sequence (collections.abc.Sequence)

Collezioni indicizzate e ordinate:

- Supportano `__getitem__()`, `index()`, `count()`.
- **Sottoclassi principali:** `list`, `tuple`, `str`.

```
from collections.abc import Sequence
print(isinstance([1, 2, 3], Sequence)) # True
```

Metodi Disponibili

Metodo	Parametri	Ritorno	Descrizione
<code>__getitem__()</code>	<code>self, index</code>	Elemento	Restituisce l'elemento all'indice specificato.
<code>__len__()</code>	<code>self</code>	<code>int</code>	Restituisce la lunghezza della sequenza.
<code>__contains__()</code>	<code>self, item</code>	<code>bool</code>	Controlla se un elemento è nella sequenza.
<code>index()</code>	<code>self, item, start=0, end=-1</code>	<code>int</code>	Restituisce l'indice della prima occorrenza.
<code>count()</code>	<code>self, item</code>	<code>int</code>	Conta il numero di occorrenze dell'elemento.

Esempio

```
from collections.abc import Sequence
```

```
class MySequence(Sequence):
    def __init__(self, data):
        self.data = data

    def __getitem__(self, index):
        return self.data[index]
```

```

def __len__(self):
    return len(self.data)

def index(self, item, start=0, end=None):
    end = end if end is not None else len(self.data)
    return self.data.index(item, start, end)

def count(self, item):
    return self.data.count(item)

my_sequence = MySequence([10, 20, 30, 10])
print(isinstance(my_sequence, Sequence)) # True

print(my_sequence[1]) # 20
print(len(my_sequence)) # 4
print(my_sequence.count(10)) # 2
print(my_sequence.index(30)) # 2

```

MutableSequence (collections.abc.MutableSequence)

Versione mutabile di Sequence:

- **Metodi extra:** append(), insert(), pop().
- **Sottoclasse principale:** list.

```

from collections.abc import MutableSequence
print(isinstance([], MutableSequence)) # True

```

Metodi Disponibili

Metodo	Parametri	Ritorno	Descrizione
<code>__setitem__()</code>	<code>self, index, value</code>	None	Modifica il valore all'indice specificato.
<code>__delitem__()</code>	<code>self, index</code>	None	Elimina l'elemento all'indice specificato.
<code>insert()</code>	<code>self, index, item</code>	None	Inserisce un elemento all'indice specificato.

append()	self, item	None	Aggiunge un elemento alla fine della lista.
pop()	self, index=-1	Element o	Rimuove e restituisce l'elemento all'indice specificato.
remove()	self, item	None	Rimuove la prima occorrenza di un elemento.

Esempio

```
from collections.abc import MutableSequence
```

```
class MyMutableSequence(MutableSequence):
```

```
    def __init__(self, data):
        self.data = list(data)
```

```
    def __getitem__(self, index):
        return self.data[index]
```

```
    def __setitem__(self, index, value):
        self.data[index] = value
```

```
    def __delitem__(self, index):
        del self.data[index]
```

```
    def insert(self, index, item):
        self.data.insert(index, item)
```

```
    def __len__(self):
        return len(self.data)
```

```
my_mut_seq = MyMutableSequence([1, 2, 3])
```

```
my_mut_seq.append(4)
```

```
my_mut_seq[1] = 10
```

```
del my_mut_seq[2]
```

```
print(my_mut_seq.data) # [1, 10, 4]
```

Set (collections.abc.Set)

Insiemi matematici, senza duplicati:

- Implementa `__contains__()`, `union()`, `intersection()`.

- **Sottoclasse principale:** set.

```
from collections.abc import Set
print(isinstance({1, 2, 3}, Set)) # True
```

La classe Set è un'astrazione per gli insiemi immutabili. Non può essere istanziata direttamente, ma definisce i metodi comuni a tutte le classi che la implementano, come frozenset.

Metodi della classe Set

Metodo	Parametri	Tipo di ritorno	Funzionalità
<code>__contains__(self, item)</code>	<code>item</code>	<code>bool</code>	Verifica se l'elemento è presente nel set.
<code>__iter__(self)</code>	-	<code>iterator</code>	Restituisce un iteratore sugli elementi del set.
<code>__len__(self)</code>	-	<code>int</code>	Restituisce il numero di elementi nel set.
<code>isdisjoint(self, other)</code>	<code>other: Set</code>	<code>bool</code>	Restituisce True se i due insiemi non hanno elementi in comune.
<code>issubset(self, other)</code>	<code>other: Set</code>	<code>bool</code>	Verifica se il set è un sottoinsieme di other.
<code>issuperset(self, other)</code>	<code>other: Set</code>	<code>bool</code>	Verifica se il set è un sovrainsieme di other.
<code>union(self, *others)</code>	<code>others: Iterable</code>	<code>Set</code>	Restituisce un nuovo set con l'unione di self e others.
<code>intersection(self, *others)</code>	<code>others: Iterable</code>	<code>Set</code>	Restituisce l'intersezione tra self e others.
<code>difference(self, *others)</code>	<code>others: Iterable</code>	<code>Set</code>	Restituisce la differenza tra self e others.
<code>symmetric_difference(self, other)</code>	<code>other: Set</code>	<code>Set</code>	Restituisce gli elementi che sono in uno solo dei due set.

Esempio d'uso di Set

```
frozenset_a = frozenset({1, 2, 3, 4})
frozenset_b = frozenset({3, 4, 5, 6})

print(frozenset_a.isdisjoint(frozenset_b)) # False
print(frozenset_a.issubset(frozenset_b))   # False
print(frozenset_a.union(frozenset_b))      # frozenset({1, 2, 3, 4, 5, 6})
print(frozenset_a.intersection(frozenset_b)) # frozenset({3, 4})
```

MutableSet (collections.abc.MutableSet)

Versione mutabile di Set:

- Aggiunge add(), discard().
- **Sottoclasse principale:** set.

```
from collections.abc import MutableSet
print(isinstance(set(), MutableSet)) # True
```

Metodi aggiuntivi di MutableSet

Metodo	Parametri	Tipo di ritorno	Funzionalità
add(self, item)	item	None	Aggiunge un elemento al set.
discard(self, item)	item	None	Rimuove un elemento senza generare errore se non esiste.
remove(self, item)	item	None	Rimuove un elemento e genera errore se non esiste.
pop(self)	-	elemento	Rimuove e restituisce un elemento casuale.
clear(self)	-	None	Rimuove tutti gli elementi dal set.

Esempio d'uso di MutableSet

```
my_set = {1, 2, 3}
my_set.add(4)
```



```
my_set.discard(2)
print(my_set)  # {1, 3, 4}
```

Mapping (`collections.abc.Mapping`)

La classe Mapping rappresenta un dizionario immutabile ovvero una **struttura chiave-valore non mutabile**:

- Implementa `keys()`, `values()`, `items()`.
- **Sottoclasse principale**: `dict`.

```
from collections.abc import Mapping
print(isinstance({"a": 1}, Mapping))  # True
```

Metodi della classe Mapping

Metodo	Parametri	Tipo di ritorno	Funzionalità
<code>__getitem__(self, key)</code>	<code>key</code>	valore	Restituisce il valore associato alla chiave.
<code>__iter__(self)</code>	-	iterator	Restituisce un iteratore sulle chiavi.
<code>__len__(self)</code>	-	int	Restituisce il numero di coppie chiave-valore.
<code>keys(self)</code>	-	KeysView	Restituisce una vista delle chiavi.
<code>values(self)</code>	-	ValuesView	Restituisce una vista dei valori.
<code>items(self)</code>	-	ItemsView	Restituisce una vista delle coppie chiave-valore.
<code>get(self, key, default=None)</code>	<code>key</code> , <code>default</code>	valore	Restituisce il valore della chiave o il valore di default.

Esempio d'uso di Mapping

```
from collections.abc import Mapping
```

```

class MyMapping(Mapping):
    def __init__(self, data):
        self._data = data

    def __getitem__(self, key):
        return self._data[key]

    def __iter__(self):
        return iter(self._data)

    def __len__(self):
        return len(self._data)

my_map = MyMapping({'a': 1, 'b': 2})
print(my_map['a']) # 1
print(list(my_map.keys())) # ['a', 'b']

```

MutableMapping (collections.abc.MutableMapping)

Versione mutabile di Mapping:

- Aggiunge `__setitem__()`, `pop()`.
- **Sottoclasse principale:** `dict`.

```

from collections.abc import MutableMapping
print(isinstance({}, MutableMapping)) # True

```

MutableMapping estende Mapping e aggiunge metodi mutabili.

Metodi aggiuntivi di MutableMapping

Metodo	Parametri	Tipo di ritorno	Funzionalità
<code>__setitem__(self, key, value)</code>	key, value	None	Imposta una coppia chiave-valore.
<code>__delitem__(self, key)</code>	key	None	Rimuove una chiave.
<code>pop(self, key, default=None)</code>	key, default	valore	Rimuove e restituisce il valore associato alla chiave.

<code>popitem(self)</code>	-	(key, value)	Rimuove e restituisce un elemento casuale.
<code>clear(self)</code>	-	None	Rimuove tutti gli elementi dal dizionario.

Esempio d'uso di MutableMapping

```
python
CopyEdit
from collections.abc import MutableMapping

class MyMutableMapping(MutableMapping):
    def __init__(self):
        self._data = {}

    def __getitem__(self, key):
        return self._data[key]

    def __setitem__(self, key, value):
        self._data[key] = value

    def __delitem__(self, key):
        del self._data[key]

    def __iter__(self):
        return iter(self._data)

    def __len__(self):
        return len(self._data)

my_dict = MyMutableMapping()
my_dict['a'] = 10
my_dict['b'] = 20
print(my_dict['a']) # 10
del my_dict['b']
print(list(my_dict.keys())) # ['a']
```

Riepilogo della gerarchia Iterable

Classe	Descrizione	Esempi
Iterable	Oggetto iterabile	list, tuple, set, dict
Collection	Estende Iterable, aggiunge len()	list, dict, set
Sequence	Collezioni ordinate e indicizzabili	list, tuple, str
MutableSequence	Versione mutabile di Sequence	list
Set	Insiemi matematici, no duplicati	set, frozenset
MutableSet	Versione mutabile di Set	set
Mapping	Struttura chiave-valore	dict, defaultdict
MutableMapping	Versione mutabile di Mapping	dict

In Python, ogni tipo di collezione ha una classe di appartenenza e una serie di metodi implementati che permettono di manipolare i dati contenuti al loro interno. Qui ti fornirò una panoramica della classe, dei metodi principali per ciascun tipo di collezione e un esempio di utilizzo.

Liste, Tuple, Set, Dizionari

1. List (Lista)

- **Classe di appartenenza:** list
- **Metodi principali:**
 - `append(x)` – Aggiunge l'elemento x alla fine della lista.
 - `extend(iterable)` – Estende la lista aggiungendo gli elementi di un iterabile.
 - `insert(i, x)` – Inserisce l'elemento x nella posizione i.
 - `remove(x)` – Rimuove la prima occorrenza di x dalla lista.
 - `pop([i])` – Rimuove e restituisce l'elemento in posizione i (se non specificato, rimuove l'ultimo elemento).
 - `sort()` – Ordina la lista in ordine crescente.
 - `reverse()` – Inverte la lista.

- `index(x)` – Restituisce l'indice della prima occorrenza di x.

Esempio di codice:

```
# Creazione di una lista
lista = [1, 2, 3, 4]

# Metodo append: aggiunge un elemento alla fine
lista.append(5)

# Metodo insert: inserisce un elemento in una posizione specifica
lista.insert(2, 10) # Inserisce 10 alla posizione 2

# Metodo remove: rimuove la prima occorrenza di 4
lista.remove(4)

# Metodo pop: rimuove e restituisce l'ultimo elemento
elemento = lista.pop()

# Metodo sort: ordina la lista
lista.sort()

# Metodo reverse: inverte la lista
lista.reverse()

print(lista)
```

2. Tuple (Tupla)

- **Classe di appartenenza:** tuple
- **Metodi principali:**
 - `count(x)` – Conta quante volte l'elemento x appare nella tupla.
 - `index(x)` – Restituisce l'indice della prima occorrenza di x.

Esempio di codice:

```
# Creazione di una tupla
tupla = (1, 2, 3, 4, 2)

# Metodo count: conta quante volte 2 appare nella tupla
conta = tupla.count(2)

# Metodo index: trova l'indice della prima occorrenza di 3
```

```
indice = tupla.index(3)

print(f"Numero di occorrenze di 2: {conta}")
print(f"Indice del primo 3: {indice}")
```

3. Set (Insieme)

- **Classe di appartenenza:** set
- **Metodi principali:**
 - `add(x)` – Aggiunge l'elemento x all'insieme.
 - `remove(x)` – Rimuove l'elemento x dall'insieme, solleva un errore se non trovato.
 - `discard(x)` – Rimuove l'elemento x se presente, non solleva errore se non trovato.
 - `pop()` – Rimuove e restituisce un elemento qualsiasi dall'insieme.
 - `clear()` – Rimuove tutti gli elementi dall'insieme.
 - `union(other)` – Restituisce l'unione di due insiemi.
 - `intersection(other)` – Restituisce l'intersezione di due insiemi.
 - `difference(other)` – Restituisce la differenza tra due insiemi.
 - `issubset(other)` – Verifica se l'insieme è un sottoinsieme di other.

Esempio di codice:

```
# Creazione di un insieme
insieme = {1, 2, 3, 4}

# Metodo add: aggiunge un elemento
insieme.add(5)

# Metodo remove: rimuove un elemento (solleva errore se non
presente)
insieme.remove(4)

# Metodo discard: rimuove un elemento senza sollevare errore
insieme.discard(10) # Non esiste, ma non solleva errore

# Metodo pop: rimuove un elemento a caso
elemento = insieme.pop()

# Metodo union: unione di due insiemi
altro_insieme = {3, 4, 5, 6}
unione = insieme.union(altro_insieme)
```

```
print(f"Unione degli insiemi: {unione}")
```

4. Dictionary (Dizionario)

- **Classe di appartenenza:** dict
- **Metodi principali:**
 - `get(key)` – Restituisce il valore associato alla chiave `key` (se non trovata, restituisce `None` o un valore predefinito).
 - `keys()` – Restituisce tutte le chiavi del dizionario.
 - `values()` – Restituisce tutti i valori del dizionario.
 - `items()` – Restituisce tutte le coppie chiave-valore come tuple.
 - `update(other)` – Aggiunge gli elementi di `other` al dizionario corrente.
 - `pop(key)` – Rimuove e restituisce l'elemento associato alla chiave `key`.
 - `popitem()` – Rimuove e restituisce l'ultima coppia chiave-valore inserita.
 - `clear()` – Rimuove tutte le coppie chiave-valore dal dizionario.

Esempio di codice:

```
# Creazione di un dizionario
dizionario = {"nome": "Mario", "età": 30}

# Metodo get: restituisce il valore associato alla chiave 'nome'
nome = dizionario.get("nome")

# Metodo keys: restituisce tutte le chiavi
chiavi = dizionario.keys()

# Metodo values: restituisce tutti i valori
valori = dizionario.values()

# Metodo items: restituisce tutte le coppie chiave-valore
coppie = dizionario.items()

# Metodo update: aggiorna il dizionario con nuove coppie
chiave-valore
dizionario.update({"sesso": "M"})

# Metodo pop: rimuove e restituisce la chiave 'età'
età = dizionario.pop("età")

# Metodo popitem: rimuove l'ultima coppia chiave-valore
```

```
ultima_coppia = dizionario.popitem()

print(f"Nome: {nome}")
print(f"Chiavi: {chiavi}")
print(f"Valori: {valori}")
print(f"Coppie chiave-valore: {coppie}")
```